

ARG-Test: Auditable Requirement-Driven Black-Box Test Generation with Structured LLM Reasoning and Contract Verification

👤 Team 7



2351441 许奕
2351039 王相
2350283 康凤轩
2352746 张洛梧
2350217 陈奕玮

Problem and Scope

"What
Did We
Do
?"



Field: AI-enhanced black-box dynamic testing

Input: Natural-language system requirements

Output: Auditable and structured test cases

Core Objective: Given natural-language requirements, we generate auditable black-box test cases with structured reasoning and verification.

Requirement Analysis and Input

Scenarios: E-commerce and business-platform rules

Dataset Split:

Development Set = 50 requirements

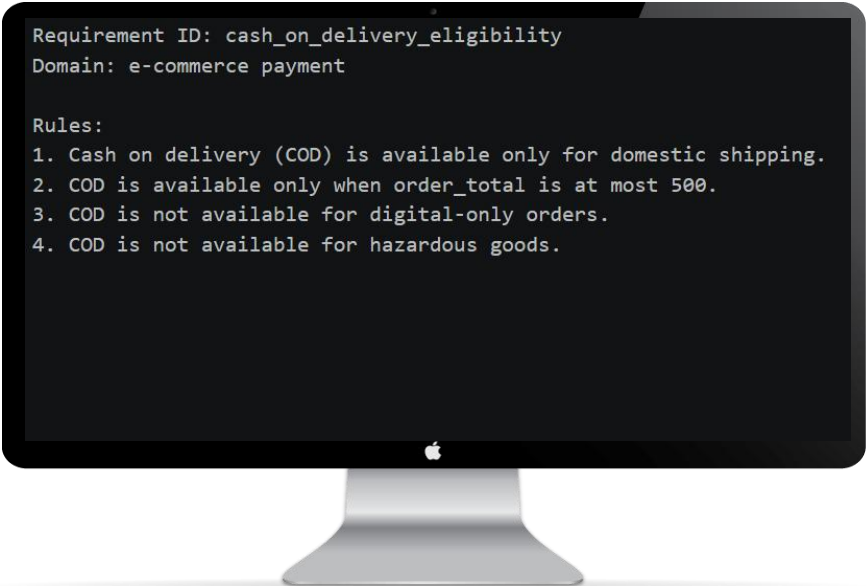
Test Set = 16 requirements

Requirement Categories:

Business Rules

Input Validation

Workflow State Transitions

A computer monitor with a silver stand and a black bezel. The screen is black with white text. The text displays requirement analysis data for 'cash_on_delivery_eligibility' in the 'e-commerce payment' domain, listing four rules.

```
Requirement ID: cash_on_delivery_eligibility
Domain: e-commerce payment

Rules:
1. Cash on delivery (COD) is available only for domestic shipping.
2. COD is available only when order_total is at most 500.
3. COD is not available for digital-only orders.
4. COD is not available for hazardous goods.
```

Tool Artifact and Prompt Design

Core Model: Qwen3.5-Flash

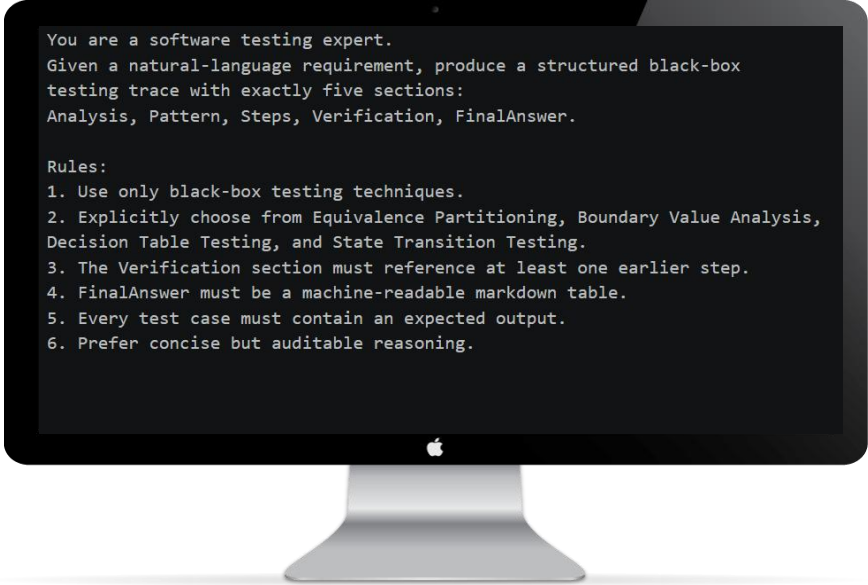
Prompt Family Approach:

System Prompt: Defines testing assistant behavior

Generation Prompt: Requests a structured reasoning trace

Repair Prompt: Fixes weak/failed outputs based on feedback

Baseline Prompt: Plain LLM baseline for comparison

A computer monitor with a silver stand and a black bezel. The screen is dark and displays white text in a monospaced font. The text is a system prompt for a software testing expert, defining the role, the task of producing a structured black-box testing trace, and a list of six rules to follow.

```
You are a software testing expert.  
Given a natural-language requirement, produce a structured black-box  
testing trace with exactly five sections:  
Analysis, Pattern, Steps, Verification, FinalAnswer.  
  
Rules:  
1. Use only black-box testing techniques.  
2. Explicitly choose from Equivalence Partitioning, Boundary Value Analysis,  
Decision Table Testing, and State Transition Testing.  
3. The Verification section must reference at least one earlier step.  
4. FinalAnswer must be a machine-readable markdown table.  
5. Every test case must contain an expected output.  
6. Prefer concise but auditable reasoning.
```

Pipeline and Architecture

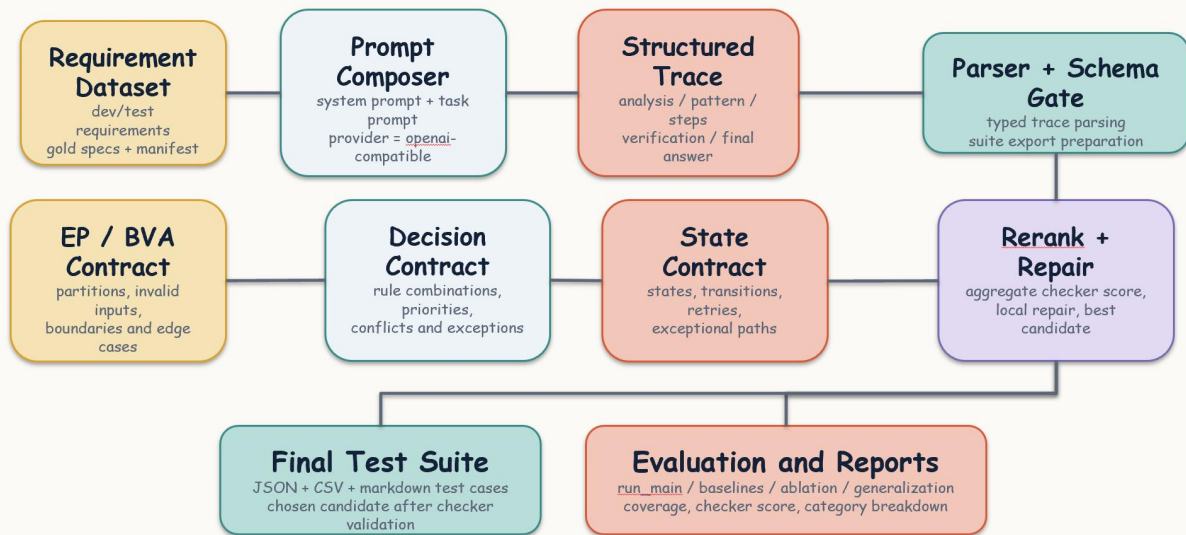
ARG-Test Architecture

Requirement-driven black-box test generation with structured reasoning, checker validation, reranking, and repair.

Model: Qwen3.5-Flash

Candidates: 3

Output: JSON / CSV / MD



Complete Pipeline Workflow:

1. Structured Trace Generation
2. Parser
3. Checker Suite (Contract Verification)
4. Reranking
5. Repair Loop
6. Final Export

Generated Output Example

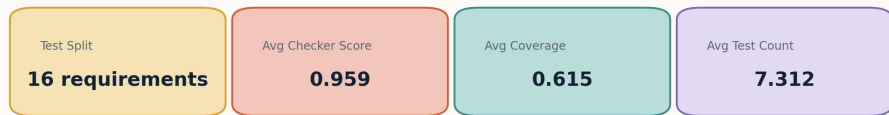
Test ID	Technique	Requirement Target	Preconditions	Input	Expected Output	Covered Item	Priority	Checker Status
T01	EP	Rule 1	Parcel Arrival	None	Order Status = READY_FOR_PICKUP	Workflow Start	High	Pending
T02	STT/EP	Rule 2	Order = READY	Code=Valid, Time=Within	Order Status = PICKED_UP	Successful Pickup	High	Pending
T03	STT/BVA	Rule 3	Order = READY	Time=Expired	Order Status = EXPIRED	Expiration Flow	High	Pending
T04	STT	Rule 4	Order = EXPIRED	Staff Action=Return	Order Status = RETURN_IN_TRANSIT	Post-Expired	Medium	Pending
T05	STT	Rule 5	Order = READY/PICKED	Damage Report=True	Order Status = HOLD_FOR_REVIEW	Damage Handling	High	Pending
T06	BVA	Rule 8	Order = READY	Attempt_Count=4	Order Status = HOLD_FOR_REVIEW	Attempt Limit	High	Pending
T07	STT	Rule 6	Order = PICKED_UP	Action=BackToReady	Error/Status Remains PICKED_UP	Illegal Reverse	Medium	Pending
T08	STT	Rule 7	Order = EXPIRED	Action=Pickup	Error/Status Remains EXPIRED	Illegal Late Pickup	Medium	Pending
T09	BVA	warehouse_pickup_order_workflow	None	repaired boundary input	boundary behavior verified	boundary case (repair)	Medium	repaired

Outputs are structured test suites, not free-form text.

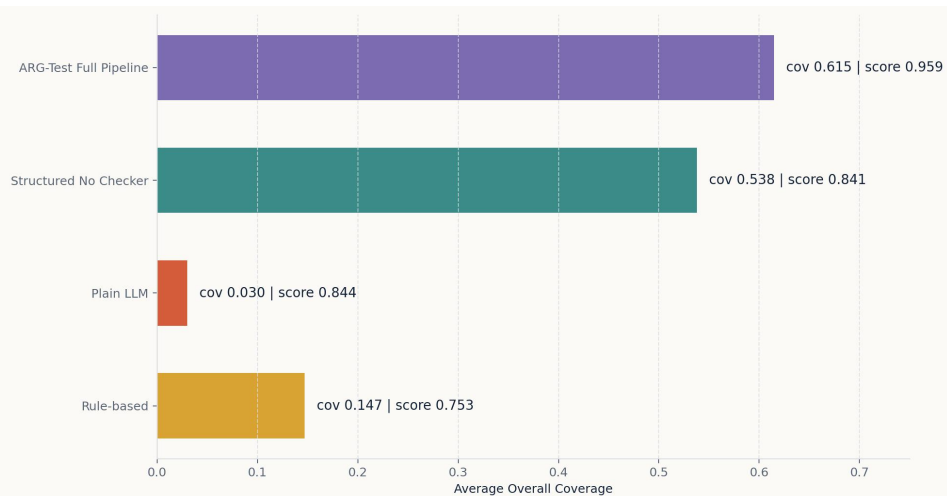
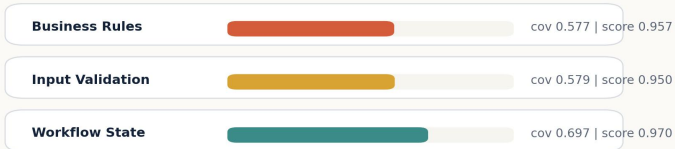
Explicit fields include:

Testing Technique Concrete Input Expected Output Covered Requirement Item

Main Result and Baselines



Category Coverage



Our Full Pipeline Results:

Average Checker Score = 0.959

Average Overall Coverage = 0.615

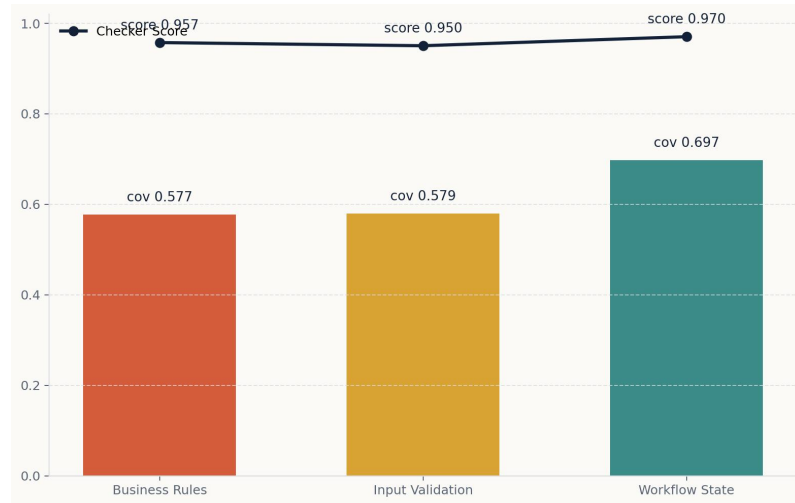
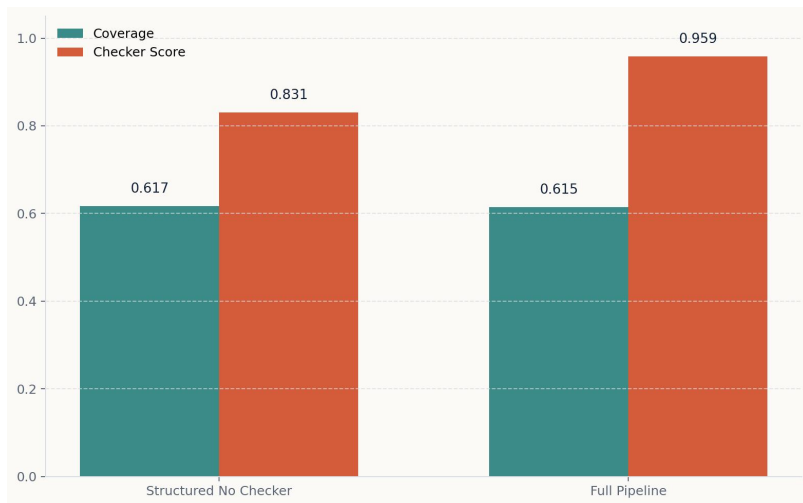
Comparison with Baselines:

Rule-based Baseline Coverage = 0.147

Plain LLM Baseline Coverage = 0.030

Structured (No Checker) Coverage = 0.538

Verification, Ablation, and Generalization



Ablation (Why do we need Checker & Repair?):

The checker-repair stage significantly boosts the checker score and reliability, ensuring test cases are auditable.

Generalization across Categories (Coverage):

Business Rules: 0.577

Input Validation: 0.579

Workflow State: 0.697

Team-AI Interaction and Revision

Problem in Early Version	What We Changed (Revision)
Unstructured, free-form text generation	Introduced structured reasoning traces
AI hallucinations and missing rules	Added parsing & contract checkers
Weak or failed test cases generated	Implemented reranking and repair loops

Conclusion

Success: ARG-Test shows that structured LLM reasoning + contract verification produces more auditable and higher-coverage testing artifacts.

Outperforms both prompt-only baselines and traditional rule-based methods.

Limitations & Future Work:

Extend the input branch from natural-language to source codebase.

Automate the execution of generated scripts.

Requirement-Driven



Structured Reasoning



High Coverage



Fully Auditable

